

คู่มืออธิบายโค้ด

โปรเจกต์ระบบเก็บเกรดนักศึกษา (TestPHP) — db.php, index.php, students.php, subjects.php, grades.php, report.php

เอกสารนี้อธิบายโค้ดจริงในโปรเจกต์ `C:\xampp\htdocs\TestPHP` ของคุณที่ละไฟล์ ทีละส่วน แบบละเอียด เพื่อให้เข้าใจว่าระบบทำงานอย่างไรตั้งแต่ต้นจนจบ เหมาะสำหรับอ่านควบคู่กับการเปิดไฟล์จริงในโปรแกรมแก้ไขโค้ด (VS Code)

สารบัญ

0. ภาพรวมระบบและแผนผังหน้าเว็บ
1. โครงสร้างฐานข้อมูลที่ระบบใช้งาน
2. db.php — ไฟล์เชื่อมต่อฐานข้อมูล
3. index.php — หน้าแรก (แดชบอร์ดภาพรวม)
4. students.php — จัดการข้อมูลนักศึกษา
5. subjects.php — จัดการข้อมูลวิชา
6. grades.php — บันทึกเกรด
7. report.php — รายงาน GPA
8. รูปแบบโค้ดที่ใช้ซ้ำในทุกไฟล์ (CRUD Pattern)
9. ฟังก์ชัน/เทคนิค PHP ที่ควรรู้จากโปรเจกต์นี้
10. ข้อสังเกตเรื่องความปลอดภัย

0. ภาพรวมระบบและแผนผังหน้าเว็บ

โปรเจกต์นี้คือ ระบบเก็บเกรดนักศึกษา ประกอบด้วย 6 ไฟล์ที่ทำงานร่วมกัน:

ไฟล์	หน้าที่
db.php	เชื่อมต่อฐานข้อมูล MySQL — ถูกเรียกใช้ (require) จากทุกไฟล์อื่น
index.php	หน้าแรก แสดงสถิติภาพรวม (จำนวนนักศึกษา/วิชา/เกรด) และรายชื่อนักศึกษาล่าสุด
students.php	เพิ่ม/แก้ไข/ลบ/แสดงรายชื่อนักศึกษา
subjects.php	เพิ่ม/แก้ไข/ลบ/แสดงรายวิชา
grades.php	บันทึกเกรดของนักศึกษาแต่ละคนในแต่ละวิชา พร้อมคำนวณเกรดอัตโนมัติจากคะแนน
report.php	แสดงรายงานผลการเรียนและ GPA ของนักศึกษาแต่ละคน แยกตามภาคเรียน

ความสัมพันธ์ระหว่างไฟล์: ทุกไฟล์ (ยกเว้น db.php เอง) เริ่มต้นด้วย `require 'db.php';` เพื่อขอใช้ตัวแปร `$conn` (การเชื่อมต่อฐานข้อมูล) จากนั้นแต่ละไฟล์จะดึง/บันทึกข้อมูลตามหน้าที่ของตัวเอง แล้วแสดงผลเป็นหน้าเว็บ HTML ที่มีแถบเมนูนำทาง (navbar) เหมือนกันทุกหน้า เชื่อมโยงไปมาระหว่าง 4 เมนูหลัก: นักศึกษา, วิชา, บันทึกเกรด, รายงาน GPA

1. โครงสร้างฐานข้อมูลที่ระบบใช้งาน

จากการอ่านคำสั่ง SQL ในทุกไฟล์ สรุปได้ว่าฐานข้อมูลชื่อ `testphp` มี 3 ตารางหลัก ดังนี้ (คอลัมน์ที่พบว่าถูกใช้งานจริงในโค้ด):

ตาราง student

คอลัมน์	ความหมาย
student_id	รหัสอ้างอิงภายใน (ใช้เรียงลำดับใน index.php ด้วย ORDER BY student_id DESC)
student_code	รหัสนักศึกษา (เช่น 6900001) ใช้เป็นรหัสหลักในการอ้างอิงข้ามตาราง
student_fname_th, student_sname_th	ชื่อ-นามสกุลภาษาไทย
student_fname_eng, student_sname_eng	ชื่อ-นามสกุลภาษาอังกฤษ
class_room	ห้องเรียน
email	อีเมล

ตาราง subject

คอลัมน์	ความหมาย
subject_code	รหัสวิชา (ใช้เป็นคีย์หลักในการอ้างอิง)
subject_name	ชื่อวิชา
credits	จำนวนหน่วยกิต (ใช้คำนวณ GPA)

ตาราง grade

คอลัมน์	ความหมาย
grade_id	รหัสอ้างอิงของแถวเกรด (Primary Key)
student_code	Foreign Key อ้างอิงไปที่ student
subject_code	Foreign Key อ้างอิงไปที่ subject
score	คะแนนดิบ (0-100)
grade	ผลเกรดตัวอักษร (A, B+, B, ... F) คำนวณจาก score

grade_point	แต้มเกรด (4.0, 3.5, ... 0.0) คำนวณจาก score
semester	ภาคเรียน (1, 2, 3)
academic_year	ปีการศึกษา (พ.ศ.)

ความสัมพันธ์ (Relationship): student และ subject เป็นตารางหลัก (master data) ส่วน grade เป็นตารางที่เชื่อมทั้งสองเข้าด้วยกัน (junction table) — นักศึกษา 1 คนมีได้หลายเกรด และวิชา 1 วิชาก็ถูกใช้ในหลายเกรดของนักศึกษาหลายคนได้ ความสัมพันธ์แบบนี้เรียกว่า **Many-to-Many** โดยมีตาราง grade เป็นตัวกลาง และตาราง grade น่าจะมี UNIQUE KEY ผสมระหว่าง (student_code, subject_code, semester, academic_year) เพราะใน grades.php ใช้คำสั่ง `ON DUPLICATE KEY UPDATE` ซึ่งจะทำงานก็ต่อเมื่อมี unique key ซ้ำกันเกิดขึ้นเท่านั้น

2. db.php — ไฟล์เชื่อมต่อฐานข้อมูล

db.php

```
<?php
$host = 'localhost';
$dbname = 'testphp';
$user = 'root';
$pass = ''; // XAMPP ค่าเริ่มต้นรหัสผ่านว่าง

$conn = new mysqli($host, $user, $pass, $dbname);
$conn->set_charset('utf8mb4');

if ($conn->connect_error) {
    die('เชื่อมต่อ DB ไม่ได้: ' . $conn->connect_error);
}
```

บรรทัด 3-6: กำหนดค่าคงที่สำหรับเชื่อมต่อฐานข้อมูล เก็บไว้ในตัวแปรแยกต่างหาก (ไม่เขียนแทรกลงในคำสั่งเชื่อมต่อตรงๆ) ทำให้แก้ไขค่าภายหลังได้ง่าย เช่น ถ้าย้ายเซิร์ฟเวอร์ ก็แค่แก้ค่า `$host`

บรรทัดที่ 8: `new mysqli(...)` คือการเชื่อมต่อฐานข้อมูลแบบ **Object-Oriented style** (ต่างจากคู่มือทั่วไปที่มักสอนแบบ `mysqli_connect()` ซึ่งเป็น procedural style) ทั้งสองแบบทำงานเหมือนกัน แต่แบบ object จะเรียกใช้คำสั่งต่างๆ ผ่าน `->` แทน เช่น `$conn->query(...)` แทนที่จะเป็น `mysqli_query($conn, ...)`

บรรทัดที่ 9: `set_charset('utf8mb4')` สั่งให้การสื่อสารกับฐานข้อมูลใช้ชุดอักขระ utf8mb4 ซึ่งรองรับภาษาไทยและอีโมจิได้ถูกต้อง **สำคัญมาก** — ถ้าลืมบรรทัดนี้ ภาษาไทยที่บันทึก/แสดงผลอาจกลายเป็นตัวอักษรแปลกๆ (เครื่องหมายคำถามหรือสัญลักษณ์มั่วๆ)

บรรทัดที่ 11-13: ตรวจสอบว่าการเชื่อมต่อมี error หรือไม่ ถ้ามีให้ `die()` หยุดการทำงานทันทีพร้อมแสดงข้อความ error ที่ชัดเจน แทนที่จะปล่อยให้หน้าเว็บพังแบบไม่รู้สาเหตุ

ไฟล์นี้ไม่มีการแสดงผล HTML ใดๆ เลย มีหน้าที่แค่ "เตรียมตัวแปร `$conn` ให้พร้อมใช้งาน" เท่านั้น ไฟล์อื่นทุกไฟล์จะ `require 'db.php';` เพื่อดึงตัวแปร `$conn` นี้ไปใช้ต่อ

3. index.php — หน้าแรก (แดชบอร์ดภาพรวม)

index.php (ส่วนหัว)

```
<?php require 'db.php'; ?>
<!DOCTYPE html>
...
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

ไฟล์นี้ทั้งไฟล์มีแค่ 1 บรรทัดที่เป็น PHP ล้วนๆ ด้านบน (require db.php) ที่เหลือเกือบทั้งหมดเป็น HTML ธรรมดาที่ใช้ Bootstrap (โหลดผ่าน CDN) ทำให้หน้าตาสวยงามและ responsive โดยไม่ต้องเขียน CSS เอง

index.php (ส่วนสถิติ)

```
$stats = [
    ['label' => 'นักศึกษาทั้งหมด', 'color' => 'primary', 'sql' => 'SELECT COUNT(*) FROM student'],
    ['label' => 'วิชาทั้งหมด', 'color' => 'success', 'sql' => 'SELECT COUNT(*) FROM subject'],
    ['label' => 'เกรดที่บันทึก', 'color' => 'warning', 'sql' => 'SELECT COUNT(*) FROM grade'],
];
foreach ($stats as $s):
    $count = $conn->query($s['sql'])->fetch_row()[0];
?>
<div class="col-md-4">
    ...
    <h2 class="fw-bold text-= $s['color'] ?&gt;"&gt;<?= $count ?&gt;&lt;/h2&gt;
    &lt;p class="text-muted mb-0"&gt;<?= $s['label'] ?&gt;&lt;/p&gt;
    ...
&lt;/div&gt;
&lt;?php endforeach; ?&gt;</pre
```

นี่คือเทคนิคที่ฉลาดมาก: แทนที่จะเขียนโค้ด 3 ชุดซ้ำกันสำหรับ 3 การ์ดสถิติ (นักศึกษา/วิชา/เกรด) ผู้เขียนสร้าง **array ของ array** (เรียกว่า "array of associative arrays") เก็บทั้ง label, สี, และคำสั่ง SQL ของแต่ละการ์ดไว้ด้วยกัน แล้วใช้ `foreach` วนลูปสร้าง HTML การ์ดทีละใบจาก array เดียวกัน

`$conn->query($s['sql'])->fetch_row()[0]` อ่านจากขวาไปซ้ายได้ดังนี้: รันคำสั่ง SQL (เช่น `SELECT COUNT(*) FROM student`) → ได้ผลลัพธ์กลับมา → `fetch_row()` ดึงแถวแรกออกมาเป็น indexed array (ต่างจาก `fetch_assoc()` ที่ได้ named key) → `[0]` เอาค่าตัวแรกใน array นั้น (เพราะ `COUNT(*)` มีคอลัมน์เดียว) จึงได้ตัวเลขจำนวนแถวออกมาตรงๆ

ถ้าอนาคตอยากเพิ่มการ์ดสถิติใหม่ (เช่น "อาจารย์ทั้งหมด") แค่เพิ่ม 1 บรรทัดใน array `$stats` โดยไม่ต้องเขียน HTML การ์ดใหม่เลย

index.php (ตารางนักศึกษาล่าสุด)

```

$rows = $conn->query('SELECT * FROM student ORDER BY student_id DESC LIMIT 10');
while ($r = $rows->fetch_assoc()):
?>
<tr>
    <td><?= htmlspecialchars($r['student_code']) ?></td>
    ...
    <td><a href="report.php?code=<?= urlencode($r['student_code']) ?>" class="btn btn-sm btn-
outline-primary">ดูเกรด</a></td>
</tr>
<?php endwhile; ?>

```

`ORDER BY student_id DESC LIMIT 10` คือ "เรียงจากคนที่เพิ่มล่าสุดก่อน แล้วเอาแค่ 10 คนแรก" ทำให้เห็นนักศึกษาที่เพิ่งถูกเพิ่มเข้าระบบล่าสุด

ปุ่ม "ดูเกรด" ใช้ `urlencode($r['student_code'])` เพื่อรหัสนักศึกษาไว้ก่อนใส่ใน URL — เพราะรหัสนักศึกษาอาจมีอักขระที่ URL ติความผิดได้ (เช่น ช่องว่างหรือสัญลักษณ์พิเศษ) urlencode จะแปลงอักขระเหล่านั้นให้ปลอดภัยสำหรับใส่ใน URL แล้วส่งต่อไปเปิดหน้า report.php พร้อมพารามิเตอร์ `code=...`

4. students.php — จัดการข้อมูลนักศึกษา

students.php (ฟังก์ชันสร้างรหัสนักศึกษาอัตโนมัติ)

```
function nextStudentCode($conn): string {
    $yearPrefix = substr((string)((int)date('Y') + 543), -2); // last 2 digits of Thai year
    $like = $conn->real_escape_string($yearPrefix);
    $row = $conn->query("SELECT student_code FROM student WHERE student_code LIKE '{$like}%'
ORDER BY student_code DESC LIMIT 1")->fetch_row();
    if ($row && preg_match('/^\d{7}$/', $row[0])) {
        $next = (int)$row[0] + 1;
    } else {
        $next = (int)($yearPrefix . '00001');
    }
    return str_pad($next, 7, '0', STR_PAD_LEFT);
}
```

ฟังก์ชันนี้คือหัวใจสำคัญของ students.php สร้างรหัสนักศึกษาใหม่แบบอัตโนมัติ ไม่ต้องให้ผู้ใช้พิมพ์เอง อธิบายทีละบรรทัด:

บรรทัดที่ 2: `date('Y')` คือปี ค.ศ. ปัจจุบัน บวก 543 แปลงเป็นปี พ.ศ. เช่น $2026+543=2569$ จากนั้น `substr(..., -2)` ตัดเอาแค่ 2 ตัวท้าย ได้ "69" — นี่คือตัวอย่างการใช้ปี พ.ศ. 2 หลักท้ายเป็นส่วนหนึ่งของรหัสนักศึกษา (คล้ายรูปแบบรหัสนักศึกษาจริงของมหาวิทยาลัยไทยหลายแห่ง)

บรรทัดที่ 4: ค้นหารหัสนักศึกษาที่ขึ้นต้นด้วยปีนี้ (เช่น ขึ้นต้นด้วย "69") โดยใช้ `LIKE '69%'` แล้วเรียงจากมากไปน้อย เอาแค่ตัวบนสุด (ตัวเลขที่ "มากที่สุด" ของปีนี้มีอยู่แล้ว) เพื่อจะได้รู้ตัวเลขถัดไปควรเป็นอะไร

บรรทัดที่ 5-8: ถ้าเจอรหัสเดิมและมันมีรูปแบบเป็นตัวเลข 7 หลักพอดี (`preg_match('/^\d{7}$/', ...)`) ตรวจสอบด้วย Regular Expression ว่าเป็นตัวเลขล้วน 7 หลัก) ให้บวก 1 เข้าไปเป็นรหัสถัดไป แต่ถ้ายังไม่เจอรหัสของปีนี้เลย (ยังไม่มีนักศึกษาปีนี้สักคน) ให้เริ่มต้นที่ `"69" . "00001"` คือ 6900001

บรรทัดสุดท้าย: `str_pad($next, 7, '0', STR_PAD_LEFT)` เติมเลข 0 ด้านหน้าให้ครบ 7 หลักเสมอ เช่นถ้า \$next คำนวณได้ 6900012 ก็ได้ผลลัพธ์เท่าเดิม แต่ถ้าเหลือได้ตัวเลขสั้นกว่า ก็เติม 0 ให้ครบ

จุดที่ควรรู้: ฟังก์ชันนี้ไม่มีช่องโหว่เล็กน้อยเรื่อง "race condition" — ถ้ามี 2 คนกดเพิ่มนักศึกษาพร้อมกันในเสี้ยววินาทีเดียวกัน อาจได้รหัสซ้ำกันได้ เพราะขั้นตอน "หารหัสล่าสุด" กับ "บันทึกรหัสใหม่" ไม่ได้ทำในธุรกรรมเดียวกัน (transaction) แต่สำหรับระบบขนาดเล็กที่ใช้คนเดียว ปัญหานี้แทบไม่มีทางเกิดขึ้นจริง

students.php (การลบ)

```
if (isset($_GET['delete'])) {
    $code = $conn->real_escape_string($_GET['delete']);
    $conn->query("DELETE FROM student WHERE student_code='{$code}'");
    header('Location: students.php?msg=deleted');
    exit;
}
```


เมื่อกดลิงก์ "ลบ" (ซึ่งมีรูปแบบ `students.php?delete=รหัส`) โค้ดจะทำงานตั้งแต่บรรทัดสุดท้ายของไฟล์ก่อนที่จะแสดง HTML ใดๆ

`real_escape_string()` คือการ "ใส่เครื่องหมายหนีอักขระอันตราย" ให้กับค่าที่ผู้ใช้ส่งมา ก่อนนำไปต่อในคำสั่ง SQL เพื่อป้องกัน SQL Injection (ดูรายละเอียดในหัวข้อที่ 10)

`header('Location: students.php?msg=deleted');` คือคำสั่งบอกให้ browser "เปลี่ยนหน้าไปที่ URL นี้แทน" (เรียกว่า redirect) ตามด้วย `exit;` เพื่อหยุดการทำงานของสคริปต์ทันที ไม่ให้โค้ดส่วนที่เหลือ (ซึ่งจะแสดงฟอร์มและตาราง) ทำงานซ้ำซ้อน — เทคนิคนี้เรียกว่า **Post/Redirect/Get pattern** ป้องกันปัญหาเวลาผู้ใช้กด refresh หน้าแล้วเกิดการลบ/บันทึกซ้ำโดยไม่ตั้งใจ

students.php (การบันทึก เพิ่ม/แก้ไข)

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $fields =
    ['student_code', 'student_fname_th', 'student_sname_th', 'student_fname_eng', 'student_sname_eng', 'class'];
    $data = [];
    foreach ($fields as $f) $data[$f] = $conn->real_escape_string(trim($_POST[$f] ?? ''));

    if ($_POST['action'] === 'add') {
        $sql = "INSERT INTO student (...) VALUES (...)";
        $conn->query($sql);
    } else {
        $old = $conn->real_escape_string($_POST['old_code']);
        $sql = "UPDATE student SET ... WHERE student_code='$old'";
        $conn->query($sql);
    }
    header('Location: students.php?msg=saved');
    exit;
}
```

แทนที่จะเขียน `$_POST['student_code']`, `$_POST['student_fname_th']` ฯลฯ ทีละตัว ผู้เขียนสร้าง array `$fields` เก็บชื่อฟิลด์ทั้งหมดไว้ แล้วใช้ `foreach` วนดึงค่าจากฟอร์มและทำความสะอาด (escape) ให้ครบทุกฟิลด์ในลูบเดียว ลดโค้ดซ้ำซ้อนได้มาก

`trim($_POST[$f] ?? '')`: `trim()` ตัดช่องว่างหน้า-หลังข้อความทิ้ง ส่วน `?? ''` คือ "null coalescing operator" หมายถึง "ถ้า `$_POST[$f]` ไม่มีค่า (ไม่ถูกส่งมา) ให้ใช้ค่าว่างแทน" ป้องกัน error กรณีฟอร์มไม่ได้ส่งฟิลด์นั้นมา

ตัวแปรที่ชื่อ `action` (ส่งมาจาก `<input type="hidden" name="action" value="...">` ในฟอร์ม) ใช้บอกว่ารอบนี้ผู้ใช้กำลัง "เพิ่มใหม่" (add) หรือ "แก้ไขของเดิม" (edit) — ถ้าเป็น edit จะมีการส่ง `old_code` มาด้วยเพื่อรู้ว่าจะ UPDATE แถวไหน (เพื่อกรณีผู้ใช้แก้ไขรหัสนักศึกษาเองด้วย ทำให้ WHERE ต้องอ้างอิงรหัสเดิม ไม่ใช่รหัสใหม่ที่เพิ่งพิมพ์)

students.php (ฟอร์มฝั่ง HTML)

```
<?php $autoCode = $edit ? $edit['student_code'] : nextStudentCode($conn); ?>
<input name="student_code" class="form-control <?=$edit ? '' : 'bg-light' ?>"
      <?=$edit ? '' : 'readonly' ?> required
      value="<?=$htmlspecialchars($autoCode) ?>">
```

ถ้าคำสั่ง "แก้ไข" (มีตัวแปร \$edit) ให้แสดงรหัสนักศึกษาเดิมและอนุญาตให้แก้ไขได้ แต่ถ้าคำสั่ง "เพิ่มใหม่" ให้เรียก

`nextStudentCode($conn)` มาสร้างรหัสให้อัตโนมัติ และใส่ attribute `readonly` เพื่อไม่ให้ผู้ใช้พิมพ์ตัวเอง (ป้องกันการตั้งรหัสซ้ำที่อาจซ้ำกับคนอื่น) เทคนิค `$edit ? 'ค่า A' : 'ค่า B'` คือ "ternary operator" ทางลัดของ if-else แบบสั้นๆ ในบรรทัดเดียว ใช้บ่อยมากเวลาต้อง toggle ค่าไปตาม 2 สถานการณ์

5. subjects.php — จัดการข้อมูลวิชา

ไฟล์นี้คือโครงสร้างเรียบง่ายที่สุดในบรรดา CRUD ทั้งหมด เพราะตาราง subject มีแค่ 3 ฟิลด์ (subject_code, subject_name, credits) และไม่มีการคำนวณพิเศษใดๆ เหมือน students.php หรือ grades.php

```
if ($_POST['action'] === 'add') {  
    $conn->query("INSERT INTO subject (subject_code,subject_name,credits) VALUES  
    ('$code','$name',$credits)");  
} else {  
    $old = $conn->real_escape_string($_POST['old_code']);  
    $conn->query("UPDATE subject SET subject_code='$code',subject_name='$name',credits=$credits  
    WHERE subject_code='$old'");  
}
```

สังเกตว่า `$credits` ไม่มีเครื่องหมายคำพูดล้อมรอบในคำสั่ง SQL (ต่างจาก `'$code'` และ `'$name'` ที่มี) เพราะบรรทัดด้านบน (`$credits = (int)$_POST['credits'];`) แปลงค่าเป็นตัวเลข (integer) ไปแล้ว จึงมั่นใจได้ว่าเป็นตัวเลขล้วนๆ ไม่มีอักขระอันตรายปนมา ไม่จำเป็นต้อง escape หรือใส่เครื่องหมายคำพูดเหมือนข้อความ (string)

โครงสร้างไฟล์นี้เหมือนกับ students.php และ grades.php ทุกประการ (ลบ → บันทึก → โหมดแก้ไข → แสดงฟอร์ม → แสดงตาราง) ดูรายละเอียดรูปแบบรวมกันในหัวข้อที่ 8

6. grades.php — บันทึกเกรด

grades.php (ฟังก์ชันคำนวณเกรด)

```
function calcGrade(float $score): array {
    if ($score >= 80) return ['A', 4.0];
    if ($score >= 75) return ['B+', 3.5];
    if ($score >= 70) return ['B', 3.0];
    if ($score >= 65) return ['C+', 2.5];
    if ($score >= 60) return ['C', 2.0];
    if ($score >= 55) return ['D+', 1.5];
    if ($score >= 50) return ['D', 1.0];
    return ['F', 0.0];
}
```

`function calcGrade(float $score): array` — ส่วน `float $score` คือการระบุ "ชนิดข้อมูลที่ต้องการ" ของพารามิเตอร์ (เรียกว่า Type Hint) บอกว่าฟังก์ชันนี้ต้องการรับค่าเป็นทศนิยม ส่วน `: array` ทำนองเดียวกันคือการระบุว่าฟังก์ชันนี้จะคืนค่าเป็น array เสมอ — การระบุชนิดข้อมูลแบบนี้ช่วยให้ PHP ช่วยตรวจจับข้อผิดพลาดได้ตั้งแต่เนิ่นๆ และช่วยให้คนอ่านโค้ดเข้าใจง่ายขึ้นว่าฟังก์ชันรับ-คืนอะไร

ฟังก์ชันไล่ตรวจเงื่อนไขจากคะแนนสูงสุดลงมาต่ำสุด (80, 75, 70, ...) ทันทึที่เจอเงื่อนไขแรกที่เป็นจริง จะ `return` ออกจากฟังก์ชันทันที จึงไม่ต้องใช้ `else if` ให้ยืดยาว — เขียนแบบนี้ได้เพราะ return หยุดการทำงานของฟังก์ชันทันที ไม่มีทางไหลไปเช็คเงื่อนไขถัดไปอีก

ค่าที่ return กลับมาเป็น array ที่มี 2 ค่า [ตัวอักษรเกรด, แด้มเกรด] เช่น ['A', 4.0]

grades.php (การเรียกใช้ผ่าน list destructuring)

```
[$grade, $gp] = calcGrade($score);
```

บรรทัดนี้เรียกว่า **list destructuring** คือการ "แกะ" array ที่ได้จากฟังก์ชันออกมาใส่ตัวแปรแยกกันในบรรทัดเดียว เทียบเท่ากับการเขียนยาวๆ ว่า `$result = calcGrade($score); $grade = $result[0]; $gp = $result[1];` แต่สั้นและอ่านง่ายกว่ามาก

grades.php (การบันทึกด้วย ON DUPLICATE KEY UPDATE)

```
$conn->query("INSERT INTO grade
(student_code,subject_code,score,grade,grade_point,semester,academic_year)
VALUES ('$student_code','$subject_code',$score,'$grade',$gp,$semester,$year)
ON DUPLICATE KEY UPDATE score=$score,grade='$grade',grade_point=$gp");
```

`ON DUPLICATE KEY UPDATE` เป็นคำสั่งพิเศษของ MySQL: ปกติ `INSERT` จะเพิ่มแถวใหม่เสมอ แต่ถ้าแถวที่กำลังจะเพิ่มไป "ชนกับ" ค่าที่ต้องไม่ซ้ำกัน (unique key) ของแถวที่มีอยู่แล้ว แทนที่จะเกิด error MySQL จะเปลี่ยนไปทำ UPDATE แถวเดิมแทนโดย

อัตโนมัติ

ในบริบทนี้ หมายความว่า: ถ้านักศึกษาคนนั้นเคยมีเกรดวิชาในภาคเรียน/ปีนี้อยู่แล้ว ระบบจะ "อัปเดตคะแนนใหม่ทับของเดิม" แทนที่จะสร้างแถวซ้ำซ้อนขึ้นมาอีกแถว เป็นวิธีที่สะดวกกว่าการเขียนเช็คเองว่า "มีอยู่แล้วหรือยัง ถ้ามีให้ UPDATE ถ้าไม่มีให้ INSERT" ด้วยโค้ด PHP

grades.php (การ JOIN หลายตารางตอนแสดงผล)

```
$rows = $conn->query("
    SELECT g.*, CONCAT(s.student_fname_th, ' ', s.student_sname_th) AS sname, sub.subject_name
    FROM grade g
    LEFT JOIN student s ON s.student_code = g.student_code
    LEFT JOIN subject sub ON sub.subject_code = g.subject_code
    ORDER BY g.academic_year DESC, g.semester DESC, g.student_code
");
```

ตาราง grade เก็บแค่ "รหัส" นักศึกษาและวิชา (student_code, subject_code) ไม่ได้เก็บชื่อจริงไว้ด้วย (นี่คือหลักการ Normalization ที่อธิบายไว้ในไฟล์ข้อสอบเล่มก่อนหน้า) ดังนั้นเวลาจะแสดงผลเป็นชื่อคนอ่านเข้าใจ ต้อง **LEFT JOIN** ไปเอาชื่อจากตาราง student และ subject มาประกอบด้วย

g, **s**, **sub** คือ "alias" (ชื่อย่อ) ของแต่ละตาราง ตั้งไว้เพื่อให้เขียนสั้นลงและไม่สับสนเวลามีคอลัมน์ชื่อซ้ำกันในหลายตาราง เช่น **g.student_code** หมายถึงคอลัมน์ student_code จากตาราง grade (alias g) ส่วน **s.student_code** หมายถึงคอลัมน์เดียวกันแต่จากตาราง student (alias s)

ใช้ **LEFT JOIN** แทน **INNER JOIN** เพื่อความปลอดภัย — ถ้าเผลอลบนักศึกษาทิ้งไปแต่ยังมีเกรดเก่าเหลืออยู่ (ข้อมูลไม่สมบูรณ์) **LEFT JOIN** จะยังแสดงแถวเกรดนั้นได้ (ช่องชื่อจะว่างเป็น NULL) แทนที่จะซ่อนแถวนั้นหายไปเลยแบบ **INNER JOIN**

grades.php (JavaScript พรีวิวกเรดแบบ real-time)

```
function previewGrade(score) {
    score = parseFloat(score);
    let g = 'F';
    if (score >= 80) g = 'A';
    else if (score >= 75) g = 'B+';
    ...
    document.getElementById('preview_grade').value = isNaN(score) ? '' : g;
}
```

ฟังก์ชันนี้ถูกเรียกทุกครั้งที่ใช้พิมพ์ในช่องคะแนน (attribute **oninput="previewGrade(this.value)"** ในช่อง input คะแนน) เพื่อให้เห็น "เกรดที่คาดว่าจะได้" ทันทีโดยไม่ต้องกด submit ก่อน

สังเกตสิ่งสำคัญ: ตรรกะการตัดเกรด (80=A, 75=B+, ...) ถูกเขียนซ้ำ 2 ที่ — ครั้งแรกในฟังก์ชัน PHP **calcGrade()** (ทำงานตอนบันทึกจริงลงฐานข้อมูล) และครั้งที่สองในฟังก์ชัน JavaScript **previewGrade()** (ทำงานแค่โชว์ตัวอย่างในหน้าเว็บ ไม่ได้บันทึกอะไร) เหตุผลที่ต้องเขียนซ้ำคือ PHP ทำงานบนเซิร์ฟเวอร์ ส่วน JavaScript ทำงานบนเครื่องผู้ใช้ทันทีที่พิมพ์ — ทั้งสองเป็นคนละภาษาคนละที่ทำงาน ไม่สามารถเรียกใช้ฟังก์ชันร่วมกันข้ามฝั่งได้ จึงต้องเขียนตรรกะเดียวกันไว้ทั้ง 2 ที่ (ข้อควรระวัง: ถ้าจะแก้เกณฑ์การให้เกรด ต้องแก้ทั้ง 2 จุดนี้ให้ตรงกัน ไม่งั้นตัวเลขที่โชว์ตอนพิมพ์กับเกรดที่บันทึกจริงจะไม่ตรงกัน)

7. report.php — รายงาน GPA

report.php (ดึงเกรดของนักศึกษาที่เลือก)

```
$result = $conn->query("
    SELECT g.*, sub.subject_name, sub.credits
    FROM grade g
    LEFT JOIN subject sub ON sub.subject_code = g.subject_code
    WHERE g.student_code='$code'
    ORDER BY g.academic_year, g.semester, g.subject_code
");
while ($r = $result->fetch_assoc()) $grades[] = $r;
```

ดึงเกรดทั้งหมดของนักศึกษาค้นที่เลือกไว้ (ผ่าน dropdown ด้านบนของหน้า) พร้อม JOIN เอาชื่อวิชาและหน่วยกิตมาด้วย แล้วเก็บทุกแถวไว้ใน array `$grades` (ทีละแถวด้วย `$grades[] = $r;` ซึ่งหมายถึง "เพิ่มค่านี้ต่อท้าย array" — เครื่องหมาย `[]` ว่างๆ ตอนกำหนดค่าคือการเพิ่มสมาชิกใหม่เข้า array)

report.php (คำนวณ GPA รวม)

```
$total_gp = 0; $total_credits = 0;
foreach ($grades as $g) {
    $total_gp += $g['grade_point'] * $g['credits'];
    $total_credits += $g['credits'];
}
$gpa = $total_credits > 0 ? $total_gp / $total_credits : null;
```

นี่คือสูตรคำนวณ GPA แบบ "ถ่วงน้ำหนักด้วยหน่วยกิต" (Credit-weighted Average) ซึ่งเป็นสูตรมาตรฐานที่มหาวิทยาลัยไทยใช้จริง หลักการคือวิชาที่หน่วยกิตเยอะกว่าจะมีผลต่อ GPA มากกว่าวิชาหน่วยกิตน้อย ไม่ใช่แค่เอาแต้มเกรดมาเฉลี่ยตรงๆ

ขั้นตอน: วนอ่านทุกวิชา เอา (แต้มเกรด × หน่วยกิต) ของแต่ละวิชามาวกสะสมไว้ใน `$total_gp` พร้อมกับสะสมหน่วยกิตรวมไว้ใน `$total_credits` สุดท้าย GPA = ผลรวมแต้มถ่วงน้ำหนัก ÷ หน่วยกิตรวมทั้งหมด

เช่น ถ้าเรียน 2 วิชา: วิชา A (3 หน่วยกิต ได้เกรด 4.0) และวิชา B (1 หน่วยกิต ได้เกรด 2.0) GPA จะไม่ใช่ $(4.0+2.0)/2 = 3.0$ แต่เป็น $(4.0 \times 3 + 2.0 \times 1) / (3+1) = 14/4 = 3.5$ เพราะวิชา A มีน้ำหนักมากกว่า

`$total_credits > 0 ? ... : null` คือการป้องกันปัญหา "หารด้วยศูนย์" (ถ้านักศึกษาค้นนี้ยังไม่มีเกรดเลย `total_credits` จะเป็น 0 ซึ่งเอาไปหารไม่ได้ในทางคณิตศาสตร์) ถ้าไม่มีหน่วยกิตเลยจะให้ GPA เป็น null (ไม่มีค่า) แทน

report.php (จัดกลุ่มเกรดตามภาคเรียน)

```

$grouped = [];
foreach ($grades as $g) {
    $key = $g['academic_year'] . '_' . $g['semester'];
    $grouped[$key][] = $g;
}
foreach ($grouped as $key => $list):
    [$yr, $sem] = explode('_', $key);
    $sem_gp = array_sum(array_map(fn($g) => $g['grade_point'] * $g['credits'], $list));
    $sem_cr = array_sum(array_column($list, 'credits'));
    $sem_gpa = $sem_cr > 0 ? $sem_gp / $sem_cr : 0;
?>

```

เป้าหมายของโค้ดชุดนี้คือ "แยกเกรดทั้งหมดออกเป็นกลุ่มตามภาคเรียน/ปีการศึกษา" เพื่อแสดงผลที่ละภาคเรียนแทนที่จะแสดงเป็นตารางยาวๆ ตารางเดียว

ขั้นตอนที่ 1 (สร้างกลุ่ม): สร้าง "key" ที่ไม่ซ้ำกันสำหรับแต่ละภาคเรียน โดยเอาปีการศึกษามาต่อกับภาคเรียนด้วย underscore เช่น ปี 2568 ภาค 1 จะได้ key เป็น "2568_1" แล้วใช้ `$grouped[$key][] = $g;` เพื่อ "โยน" แถวเกรดนี้เข้ากลุ่มที่ตรงกับ key ของมัน (ถ้ายังไม่มีย่อยนี้มาก่อน PHP จะสร้าง array ว่างให้อัตโนมัติ) ผลลัพธ์คือ \$grouped จะกลายเป็น array ที่แต่ละ key คือ 1 ภาคเรียน และค่าคือ array ของเกรดทั้งหมดในภาคเรียนนั้น

ขั้นตอนที่ 2 (แกะ key กลับ): `explode('_', $key)` คือการตัด string ตามตัวคั่น (underscore) กลับออกมาเป็น array 2 ช่อง เช่น "2568_1" จะกลายเป็น `['2568', '1']` แล้ว list destructuring `[$yr, $sem]` แกะออกมาเป็นตัวแปรปีและภาคเรียนแยกกัน

ขั้นตอนที่ 3 (คำนวณ GPA เฉพาะภาคเรียนนี้): ใช้หลักการเดียวกับ GPA รวมด้านบน แต่เขียนแบบกระชับด้วยฟังก์ชัน built-in ของ PHP:

- `array_map(fn($g) => $g['grade_point'] * $g['credits'], $list)` : วนคูณ (แต้มเกรด×หน่วยกิต) ของทุกวิชาในภาคเรียนนี้ ได้ array ของผลคูณออกมา (`fn(...) => ...` คือ "arrow function" ฟังก์ชันสั้นๆ แบบไม่ระบุชื่อ เขียนแทน function เต็มรูปแบบได้เมื่อโค้ดสั้นบรรทัดเดียว)
- `array_sum(...)` : บวกค่าทุกตัวใน array นั้นรวมกัน
- `array_column($list, 'credits')` : ดึงเฉพาะค่าคอลัมน์ 'credits' ออกมาจากทุกแถวใน \$list เป็น array เดียว (ยอกว่าการเขียน foreach เพื่อดึงค่าออกมาเอง)

8. รูปแบบโค้ดที่ใช้ซ้ำในทุกไฟล์ (CRUD Pattern)

ไฟล์ `students.php`, `subjects.php` และ `grades.php` ใช้โครงสร้างเดียวกันทุกไฟล์ เมื่อเข้าใจ pattern นี้ครั้งเดียว จะอ่านทั้ง 3 ไฟล์ออกทันที:

ลำดับการทำงานเมื่อเปิดไฟล์ (บนลงล่าง):

- 1) `require 'db.php';` — เชื่อมต่อฐานข้อมูล
- 2) **เช็คความมีคำสั่งลบใหม่** — ถ้า URL มี `?delete=...` ให้ลบทันทีแล้ว redirect กลับ (จบการทำงานตรงนี้เลย ไม่ทำขั้นตอนถัดไป)
- 3) **เช็คความมีการ submit ฟอรมใหม่** — ถ้า `$_SERVER['REQUEST_METHOD'] === 'POST'` ให้บันทึก (เพิ่มใหม่หรือแก้ไข ตามค่า action) แล้ว redirect กลับ (จบการทำงานตรงนี้เลยเช่นกัน)
- 4) **เช็คว่ามีอยู่ในโหมดแก้ไขไหม** — ถ้า URL มี `?edit=...` ให้ดึงข้อมูลแถวนั้นมาเตรียมแสดงในฟอร์ม (เก็บไว้ในตัวแปร `$edit`)
- 5) **แสดง HTML** — วาดฟอร์ม (ถ้ามี `$edit` ให้เติมค่าเดิมลงช่องต่างๆ ด้วย) ตามด้วยตารางแสดงข้อมูลทั้งหมดที่มีอยู่ในระบบ

เหตุผลที่ขั้นตอน 2 และ 3 ต้อง "จบการทำงานทันที" (ด้วย `exit` หลัง `header redirect`) คือเพื่อให้ไหลต่อไปแสดง HTML ช้อนทับกับหน้าที่กำลังจะเปลี่ยนไป — ทุกครั้งที่มีการเปลี่ยนแปลงข้อมูล (เพิ่ม/ลบ/แก้ไข) ระบบจะ "เปลี่ยนหน้า" ไปยัง URL ใหม่เสมอ (มี `?msg=saved` หรือ `?msg=deleted` ต่อท้าย) ซึ่งเมื่อไปถึงหน้าใหม่ ค่า `REQUEST_METHOD` จะกลายเป็น GET (ไม่ใช่ POST) ทำให้ขั้นตอน 3 ไม่ทำงานซ้ำ ป้องกันปัญหาเกิด refresh แล้วบันทึกซ้ำโดยไม่ตั้งใจ

9. ฟังก์ชัน/เทคนิค PHP ที่ควรรู้จากโปรเจกต์นี้

คำสั่ง/เทคนิค	ความหมาย
<code>real_escape_string()</code>	ใส่เครื่องหมายหนีอักขระอันตรายในข้อความ ก่อนนำไปต่อในคำสั่ง SQL เพื่อป้องกัน SQL Injection
<code>?? ' '</code> (Null Coalescing)	"ถ้าค่าซ้ายมือไม่มีค่า/เป็น null ให้ใช้ค่าขวามือแทน" ป้องกัน error จากตัวแปรที่ไม่มีอยู่
<code>[\$a, \$b] = array(...)</code> (List Destructuring)	แกะค่าจาก array ออกมาใส่ตัวแปรหลายตัวพร้อมกันในบรรทัดเดียว
<code>fetch_assoc()</code>	ดึง 1 แถวผลลัพธ์ออกมาเป็น array ที่ key ตรงกับชื่อคอลัมน์ (เข้าถึงด้วยชื่อ)
<code>fetch_row()</code>	ดึง 1 แถวผลลัพธ์ออกมาเป็น array ที่ใช้ตัวเลข index (เข้าถึงด้วยตำแหน่ง เช่น [0])
<code>data_seek(0)</code>	ย้อนตัวชี้ผลลัพธ์กลับไปแถวแรก เพื่อวนอ่านผลลัพธ์เดิมซ้ำได้อีกครั้ง (ใช้ใน grades.php เพราะ dropdown นักศึกษา/วิชาต้องถูกวาดใหม่ทุกครั้งที่โหลดฟอร์ม)
<code>ON DUPLICATE KEY UPDATE</code>	คำสั่ง SQL ของ MySQL: ถ้า INSERT ชนกับ unique key เดิม ให้ UPDATE แถวนั้นแทนที่จะ error
<code>array_column()</code>	ดึงเฉพาะค่าคอลัมน์เดียวออกมาจากทุกแถวใน array เป็น array ใหม่
<code>array_map()</code>	นำฟังก์ชันไปใช้กับทุกสมาชิกใน array แล้วคืน array ผลลัพธ์ใหม่
<code>array_sum()</code>	บวกค่าทุกตัวใน array รวมกัน
<code>fn(\$x) => ...</code> (Arrow Function)	ฟังก์ชันสั้นแบบไม่ระบุชื่อ เขียนแทน function เต็มรูปแบบเมื่อโค้ดสั้นเพียงบรรทัดเดียว
Type Hint เช่น <code>float \$score</code> , <code>:</code> <code>array</code>	ระบุชนิดข้อมูลของพารามิเตอร์/ค่าที่ฟังก์ชันคืนกลับ ช่วยตรวจจับข้อผิดพลาดและให้อ่านโค้ดง่ายขึ้น
Post/Redirect/Get Pattern	หลังบันทึก/ลบข้อมูลสำเร็จ ให้ redirect ไปหน้าใหม่เสมอ แทนที่จะแสดงผลต่อในหน้าเดิม ป้องกันการส่งข้อมูลซ้ำตอน refresh

10. ข้อสังเกตเรื่องความปลอดภัย

โปรเจกต์นี้ใช้ `real_escape_string()` แทน **Prepared Statement** ซึ่งต่างจากแนวทางที่แนะนำในคู่มือสอบ (ที่เน้น Prepared Statement เป็นมาตรฐานที่ปลอดภัยกว่า) แต่การใช้ `real_escape_string()` ควบคุมกับการแปลงชนิดข้อมูล (เช่น `(int)`, `(float)`) กับค่าที่ควรเป็นตัวเลขเสมอ ก็ยังถือว่าป้องกัน SQL Injection ได้ในระดับที่ปลอดภัยเพียงพอ **ทราบไหมที่ไม่ลืม escape หรือ cast ค่าจากผู้ใช้อย่างแต่จุดเดียว**

จุดที่ควรระวังเป็นพิเศษถ้าจะนำโค้ดนี้ไปพัฒนาต่อ: ทุกจุดที่มีการรับค่าจาก `$_GET` หรือ `$_POST` แล้วนำไปต่อในคำสั่ง SQL โดยตรง ต้องผ่าน `real_escape_string()` (สำหรับข้อความ) หรือ `cast` เป็นตัวเลข (สำหรับตัวเลข) เสมอทุกครั้งโดยไม่มีข้อยกเว้น เพราะถ้าลืมแม้แต่จุดเดียว จุดนั้นจะกลายเป็นช่องโหว่ทันที — ข้อดีของ Prepared Statement ที่แนะนำในคู่มือสอบคือ "บังคับ" ให้แยกค่าข้อมูลออกจากคำสั่งเสมอโดยอัตโนมัติ ไม่ต้องพึ่งความระมัดระวังของผู้เขียนโค้ดในทุกจุด จึงปลอดภัยกว่าในระยะยาวเมื่อโปรเจกต์ใหญ่ขึ้นและมีคนเขียนโค้ดหลายคน

คู่มืออธิบายจากโค้ดจริงในโปรเจกต์ TestPHP ของคุณ ณ วันที่จัดทำเอกสาร หากมีการแก้ไขโค้ดภายหลัง เนื้อหาบางส่วนอาจไม่ตรงกับโค้ดล่าสุด ควรเทียบกับไฟล์จริงเสมอ